

基于Ember数据集的恶意代码检测模型分析

2211985 李佳璐

1. 实验背景

Ember 是一个用于恶意代码检测的大型公开数据集，包含关于PE文件的多个特征。目标是从训练数据中提取特征并训练恶意代码检测模型。本实验通过实现一个神经网络模型，结合Ember数据集的特征，评估模型在检测恶意代码方面的性能。

2. 实验目标

- 分析Ember数据集中样本的PE文件特征：**提取特定特征如字节熵、直方图、导入表和PE头部信息。
- 训练恶意代码检测模型：**使用至少一种机器学习方法（如神经网络），基于特征数据训练分类器。
- 验证模型性能：**基于验证集和测试集，评估模型的准确率和泛化性能。
- 输出实验结论和可改进方向。**

3. 实验过程

1. 特征分析

(1) PE文件特征的背景

PE (Portable Executable) 文件是Windows操作系统中广泛使用的可执行文件格式，主要包括.exe和.dll文件。PE文件结构复杂且富含信息，这些特征在恶意代码检测中尤为重要。基于Ember数据集，可以提取和分析PE文件中的以下关键特征，从而为恶意代码检测提供基础。

(2) Ember数据集的PE文件特征分类

Ember数据集中的PE文件特征可以分为以下几类，每类特征分别用于分析不同方面的恶意行为模式：

字节熵特征 (Byte Entropy Features)

- 字节熵用于衡量文件中数据的分布均匀性，表示文件内容的随机性。
 - 恶意代码经常使用加密、混淆技术，这会显著提高文件的熵值。
 - 高熵区域可能对应于恶意的payload，如加壳文件或嵌入式加密数据。
- 处理方法：**将PE文件的字节内容分块，计算每个块的熵值并生成特征。

字节直方图特征 (Histogram Features)

- 统计文件中每个字节值 (0-255) 的频率，形成一个256维的特征向量。
 - 特定字节值 (如NOP指令、常见的反汇编指令) 频率异常**可能揭示恶意代码行为。
 - 混淆或加密会导致字节分布均匀化，影响直方图分布。
- 处理方法**：对PE文件的字节序列进行直方图统计，归一化后作为特征输入。

导入表特征 (Import Features)

- 记录PE文件中调用的外部API函数 (函数导入表)。
 - 恶意代码经常调用**敏感API**，如 `VirtualAlloc` (分配内存)、`WriteProcessMemory` (进程注入)、`WinExec` (执行命令) 等。
 - 分析导入表特征可以有效识别恶意行为模式。
- 处理方法**
 - 提取导入表中所有API名称，对名称进行哈希处理。
 - 生成256维的特征向量用于表示API调用分布。

PE头部特征 (PE Header Features)

- PE文件的头部包含文件的基本结构信息，如节表 (Section Table)、时间戳、文件大小等。
 - 恶意代码经常通过**修改PE头部**来隐藏自身 (如伪造时间戳、修改入口点等)。
 - 异常的节表数量或节名 (如 `.textbss` 等非标准节名) 可能表明恶意行为。
- 处理方法**：提取PE头部字段，并通过哈希处理生成特征。

特定节的特征 (Section Features)

- PE文件中节 (Section) 是存储代码、数据和资源的区域，常见的节有 `.text`、`.data`、`.rsrc` 等。
 - 恶意代码可能**添加异常节** (如 `.xyz`) 或伪造节名。
 - 节的大小和熵值可以反映恶意代码注入、混淆的程度。
- 处理方法**：提取各节的名称、大小、熵值等特征并组合成特征向量。

4. 训练恶意代码检测模型

1. 特征提取

- 字节熵 (Byte Entropy)**：利用 `byteentropy` 字段表示文件的熵特性。

```
class FirstFeature():
    def __init__(self, texts):
        self.texts = texts

    def byteentropy(self):
        entropy = [text['byteentropy'] for text in self.texts]
        return np.array(entropy)
```

从每条数据中提取 `byteentropy`，查看shape

```
entorpy = FirstFeature(texts).byteentropy()

entorpy = torch.tensor(entorpy, dtype=torch.float32)
```

- **直方图 (Histogram)** : 从 `histogram` 字段提取直方图特征。

```
numpy.bincount(x, weights=None, minlength=None)
```

- **导入表哈希 (Import Hashing)** : 对导入表中的函数名进行SHA-256哈希, 并通过余数生成256维特征向量。

```
def custom_hash(self, input_str):  
    hash_object = hashlib.sha256(input_str.encode())  
    hash_hex = hash_object.hexdigest()  
    hash_int = int(hash_hex, 16)  
    return hash_int
```

- **PE头部特征 (PE Header Features)** : 对PE文件头部字段进行哈希处理, 生成256维特征。

```
class FourFeature():  
    def __init__(self, texts):  
        self.texts = texts  
        self.header_name = self.header()  
  
    def header(self):  
        header_name = [wash(text['header']) for text in self.texts]  
        return header_name  
  
    def custom_hash(self, input_str):  
        hash_object = hashlib.sha256(input_str.encode())  
        hash_hex = hash_object.hexdigest()  
        hash_int = int(hash_hex, 16)  
        return hash_int  
  
    def hash_to_256(self):  
        name_to_256 = []  
        for header in self.header_name:  
            every_num = [self.custom_hash(single_name) % 256 for  
single_name in header]  
            name_to_256.append(every_num)  
  
        all_hash256 = []  
        for nums in name_to_256:  
            hash256 = [0] * 256  
            for num in nums:  
                hash256[num] += 1  
            all_hash256.append(hash256)  
  
        return np.array(all_hash256)
```

2. 数据处理

- 使用PyTorch将特征组合为张量 (形状为 [样本数, 1024]) , 并分为训练集、验证集和测试集。
- 标签处理为二分类问题, 分别表示恶意 (1) 和正常 (0) 。

3. 模型设计

代码中使用了PyTorch框架定义了一个简单的多层感知机 (MLP) 模型, 其主要结构如下:

- **输入层**: 接收从Ember数据集中提取的特征 (如PE文件特征) , 特征维度为 1024 。

- **隐藏层**：网络可能有多个隐藏层，使用ReLU激活函数引入非线性。
- **输出层**：用于二分类（恶意文件与正常文件），输出为 1（恶意）或 0（正常）。

```
pythonCopy codeclass MLP(torch.nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(MLP, self).__init__()
        self.fc1 = torch.nn.Linear(input_size, hidden_size)
        self.relu = torch.nn.ReLU()
        self.fc2 = torch.nn.Linear(hidden_size, output_size)
        self.softmax = torch.nn.Softmax(dim=1)
```

- **网络架构**：包括两层全连接层、批归一化（Batch Normalization）

```
def _weights_init(m):
    if isinstance(m, nn.Linear): # 只初始化线性层
        nn.init.xavier_uniform_(m.weight)
        if m.bias is not None:
            nn.init.zeros_(m.bias)
    elif isinstance(m, nn.BatchNorm1d): # 批归一化层
        nn.init.uniform_(m.weight, 0, 1)
        nn.init.zeros_(m.bias)
```

- 恶意代码分类

网络架构：PreLU 激活函数、Dropout 正则化，Softmax 分类

损失函数：交叉熵损失（CrossEntropyLoss），是二分类任务的常用损失函数

优化器：使用了 torch.optim.Adam 优化算法，这是一种优化性能较好的自适应学习率优化算法。

```
class MalwareClassifier(nn.Module):
    def __init__(self, input_dim=1024, hidden_dim=512, output_dim=2):
        super(MalwareClassifier, self).__init__()
        self.linear1 = nn.Linear(input_dim, hidden_dim)
        self.batch1 = nn.BatchNorm1d(hidden_dim)
        self.drop1 = nn.Dropout(0.5)
        self.prelu1 = nn.PReLU()

        self.linear2 = nn.Linear(hidden_dim, hidden_dim // 2)
        self.batch2 = nn.BatchNorm1d(hidden_dim // 2)
        self.drop2 = nn.Dropout(0.5)
        self.prelu2 = nn.PReLU()

        self.clas = nn.Linear(hidden_dim // 2, output_dim)
        self.sigmoid = nn.Sigmoid()
        self.soft = nn.Softmax(dim=-1)
        self.loss = nn.CrossEntropyLoss()

    def forward(self, x, label=None):
        x = torch.log10(1 + x)
        x = self.linear1(x)
        x = self.drop1(x)
        x = self.batch1(x)
        x = self.prelu1(x)

        x = self.linear2(x)
```

```

x = self.drop2(x)
x = self.batch2(x)
x = self.prelu2(x)

logits = self.clas(x)
probabilities = self.soft(logits)
predictions = torch.argmax(probabilities, dim=-1)

if label is not None:
    loss = self.loss(logits, label)
    return loss, predictions
return probabilities, predictions

```

- **优化器**: Adam优化器, 学习率为 0.0001。

```

# 初始化模型
model = MalwareClassifier(input_dim=1024, hidden_dim=512, output_dim=2)
optimizer = torch.optim.Adam(model.parameters(), lr=0.0001)
epochs = 10

```

4. 训练和验证

- 模型在CPU设备上运行, 进行10轮训练。
- 每轮输出训练集和验证集的准确率与损失。
- 训练过程中通过**多次迭代 (Epoch) 更新网络权重**, 不断降低训练损失, 同时在验证集上评估模型性能。

5. 测试模型

- 使用测试集评估模型的最终性能, 并计算准确率。

5. 验证性能

```

(.venv) PS D:\desktop\Malware\pythonProject> python ember_processing.py
torch.Size([110000, 1024])
D:\desktop\Malware\pythonProject\ember_processing.py:185: UserWarning: To copy cons
().detach() or sourceTensor.clone().detach().requires_grad_(True), rather than torch
    features = torch.tensor(features, dtype=torch.float32) # 特征张量
训练集大小: torch.Size([88000, 1024])
验证集大小: torch.Size([11000, 1024])
测试集大小: torch.Size([11000, 1024])
当前设备: cpu
Epoch 1/10, Loss: 248.8624, Train Acc: 0.8442, Val Loss: 20.1337, Val Acc: 0.9078
Epoch 2/10, Loss: 177.5349, Train Acc: 0.8950, Val Loss: 19.2932, Val Acc: 0.9117
Epoch 3/10, Loss: 156.8814, Train Acc: 0.9090, Val Loss: 15.7697, Val Acc: 0.9289
Epoch 4/10, Loss: 145.5963, Train Acc: 0.9167, Val Loss: 15.0388, Val Acc: 0.9353
Epoch 5/10, Loss: 137.9287, Train Acc: 0.9213, Val Loss: 15.0627, Val Acc: 0.9321
Epoch 6/10, Loss: 132.8176, Train Acc: 0.9246, Val Loss: 17.3088, Val Acc: 0.9204
Epoch 7/10, Loss: 127.8989, Train Acc: 0.9274, Val Loss: 16.4974, Val Acc: 0.9280
Epoch 8/10, Loss: 123.0159, Train Acc: 0.9305, Val Loss: 16.9441, Val Acc: 0.9247
Epoch 9/10, Loss: 119.7831, Train Acc: 0.9318, Val Loss: 16.6639, Val Acc: 0.9247
Epoch 10/10, Loss: 116.1002, Train Acc: 0.9347, Val Loss: 13.5969, Val Acc: 0.9427
Test Accuracy: 0.9402

```

1. 验证性能分析

(1) 损失值 (Loss)

- 验证集 (Val Loss): 验证集损失值在训练中逐渐降低, 从初始的 **20.13** 降低到最终的 **13.60**。这表明模型在训练过程中能够有效优化, 并减少错误率。

(2) 验证集准确率 (Val Acc)

- 验证集准确率从第一个Epoch的 **90.78%** 提升到最后的 **94.27%**, 表明模型在验证集上的分类性能较强。
- 验证集准确率超过90%, 且损失持续下降, 显示模型的泛化能力较好。

2. 测试性能分析

- 测试集准确率 (Test Accuracy)
 - 最终测试集准确率为 **94.02%**, 与验证集性能 (94.27%) 接近, 表明模型在新数据上的预测能力较强。
 - 测试集性能与验证集性能相差较小, 说明模型没有明显的过拟合现象。

3. 结论

(1) 验证性能表现

- 验证集准确率和损失表现均良好, 说明模型具有较强的泛化能力, 可以较好地地区分恶意文件与正常文件。

(2) 测试性能表现

- 测试集准确率达到 **94.02%**, 显示模型在实际恶意文件检测任务中的表现较为优异。